Xây dựng kho tri thức từ tài liệu PDF

Trần Minh Tân · MSSV 2A202603688 · Day 06 Data Ingestion Engineering

Mục tiêu

Tìm các đoạn tài liệu liên quan đến câu hỏi và truy ngược được về trang gốc.

Dữ liệu

Giáo trình Kỹ thuật chế biến món ăn · dùng 62 trang nội dung.

Sản phẩm

Pipeline Python, kho vector Qdrant, giao diện test và bộ đánh giá 20 câu hỏi.

ĐIỂM CẦN NHỚ

Bài này tập trung vào chuẩn bị dữ liệu và tìm bằng chứng.

Giao diện hiện trả về đoạn tài liệu, chưa sinh câu trả lời bằng LLM.

LỜI TRÌNH BÀY

Em là Trần Minh Tân. Trong bài lab này, em xây dựng một pipeline để biến giáo trình PDF thành kho tri thức có thể tìm kiếm theo ý nghĩa. Khi nhập câu hỏi, hệ thống trả về các đoạn liên quan cùng trang nguồn để đối chiếu. Sau đây em trình bày dữ liệu đầu vào, từng bước xử lý, cách đánh giá và demo giao diện.

1. Khảo sát và chọn dữ liệu đầu vào

Đọc được file chưa có nghĩa mọi trang đều là dữ liệu hữu ích.

71 trang vật lý → 62 trang nội dung

Dùng trang 9–70; loại phần đầu 1–8 và trang 71 chỉ còn số trang.

4 nhóm nội dung

Tổng luận/sơ chế; phối hợp nguyên liệu/gia vị; làm chín; trình bày món ăn.

Lưu thông tin nguồn trong manifest

Tên tài liệu, quyền sử dụng, hash PDF, các trang được tính và số ký tự.

ĐIỂM CẦN NHỚ

PDF do học viên cung cấp; bìa ghi năm 2019.

Chưa có URL tải gốc. Trang 2 đề cập mục đích đào tạo/tham khảo; không tự gán giấy phép mở.

LỜI TRÌNH BÀY

Trước khi viết pipeline, em khảo sát PDF để biết có lấy được chữ hay không và trang nào có nội dung hữu ích. File có 71 trang nhưng em chỉ sử dụng 62 trang từ 9 đến 70. Em ghi các quyết định này trong manifest để người khác biết nguồn dữ liệu và có thể kiểm tra lại cách tính trang.

2. Toàn bộ quy trình hoạt động

Pipeline là chuỗi bước biến đổi dữ liệu, đầu ra bước trước trở thành đầu vào bước sau.

01 · Đọc PDF

Text từng trang

02 · Chuẩn hóa

Giữ nội dung & nguồn

03 · Chia chunk

Đoạn nhỏ có ID

04 · Embedding

Vector 384 chiều

05 · Qdrant

Lưu vector + metadata

06 · Truy xuất

Câu hỏi → top-3

ĐIỂM CẦN NHỚ

Lúc chuẩn bị: tài liệu → vector → kho lưu trữ.

Lúc truy vấn: câu hỏi → vector → tìm 3 đoạn gần nhất trong kho đã tạo.

LỜI TRÌNH BÀY

Quy trình gồm sáu bước. Đầu tiên em đọc chữ từ PDF, sau đó chuẩn hóa text và gắn nguồn, số trang. Tiếp theo em chia text thành các đoạn nhỏ, tạo embedding cho từng đoạn và lưu vào Qdrant. Khi người dùng hỏi, câu hỏi được tạo vector bằng cùng model để tìm ba đoạn liên quan nhất.

3. Đọc PDF và chuẩn hóa text

Giữ được nội dung và dấu nguồn quan trọng hơn việc làm text nhìn đẹp.

Đọc từng trang bằng pypdf

Mỗi bản ghi có document_id, source, page, text và metadata.

Chuẩn hóa có kiểm soát

Unicode NFC; gộp khoảng trắng ngang; thống nhất xuống dòng; bỏ dòng số trang đầu.

Ghi nhận lỗi trích xuất

Một số từ bị tách như “nguyên li ệu”. Giữ nguyên, chưa sửa tự động bằng suy đoán.

ĐIỂM CẦN NHỚ

Số trang luôn là trang vật lý trong PDF để đối chiếu.

Nếu đọc lỗi hoặc thiếu trang đã khai báo, pipeline ghi log và dừng.

LỜI TRÌNH BÀY

Em dùng pypdf để đọc riêng từng trang, thay vì gộp cả tài liệu thành một chuỗi. Sau đó em chuẩn hóa Unicode và khoảng trắng nhưng vẫn giữ cấu trúc dòng. Một số từ trong PDF bị tách chữ, em ghi nhận hạn chế này và chưa tự sửa vì có thể làm sai nội dung. Mỗi bản ghi luôn giữ nguồn và số trang.

4. Chunk là gì? Vì sao phải chia nhỏ?

Chunk là một đoạn nội dung dùng làm đơn vị tạo vector và truy xuất.

Không dùng cả PDF làm một kết quả

Người hỏi cần một đoạn có bằng chứng cụ thể, không phải toàn bộ 62 trang.

Chunk quá nhỏ / quá lớn

Quá nhỏ dễ mất chủ thể và điều kiện; quá lớn chứa nhiều ý, khó tập trung vào câu hỏi.

Giữ liên kết về nguồn

Mỗi chunk có ID ổn định, trang và vị trí ký tự; không chia vượt qua ranh giới trang.

ĐIỂM CẦN NHỚ

Ví dụ: một đoạn giải thích vai trò của gia vị có thể là một chunk.

Mục tiêu là đoạn đủ ngữ cảnh nhưng vẫn tập trung vào một nhóm ý.

LỜI TRÌNH BÀY

Chunk có thể hiểu là một mẩu nhỏ của tài liệu. Việc chia nhỏ giúp hệ thống trả về bằng chứng cụ thể. Tuy nhiên, nếu chia quá ngắn thì dễ mất ý; chia quá dài thì có nhiều thông tin không liên quan. Vì vậy em thử hai công thức chia và đo xem công thức nào hỗ trợ truy xuất tốt hơn.

5. Embedding biến văn bản thành vector

Vector là một dãy số giúp hệ thống so sánh độ gần nhau về ý nghĩa.

Model thật, chạy trên máy

paraphrase-multilingual-MiniLM-L12-v2 qua FastEmbed, bản ONNX quantized.

Mỗi chunk → vector 384 chiều

Model nhận text và trả về một dãy gồm 384 số; không dùng vector ngẫu nhiên.

Câu hỏi dùng cùng model

So sánh vector câu hỏi với vector chunk bằng cosine similarity.

ĐIỂM CẦN NHỚ

Cosine cao hơn nghĩa là gần hơn theo model.

Điểm tương đồng không phải xác suất câu trả lời đúng. Model vẫn có thể nhầm các đối tượng gần nghĩa.

LỜI TRÌNH BÀY

Embedding là bước chuyển đoạn văn thành một dãy số để so sánh ý nghĩa. Em sử dụng MiniLM đa ngữ chạy local và mỗi vector có 384 chiều. Câu hỏi cũng được xử lý bằng cùng model. Hệ thống dựa vào độ tương đồng cosine để xếp hạng, nhưng điểm cao không bảo đảm đoạn đó chứa đúng bằng chứng cần tìm.

6. Qdrant lưu những gì?

Vector giúp tìm kiếm; text và metadata giúp đọc kết quả và kiểm tra nguồn.

Vector + text + metadata

Lưu vector, nội dung chunk, document_id, source, page, content_group và offset.

Lưu bền vững trên ổ đĩa

Database ở .local/qdrant; có ba collection day06_A, day06_B và day06_C.

Nạp lại và cập nhật

Upsert theo stable ID chống trùng; refresh xóa điểm cũ rồi nạp lại để không giữ chunk lỗi thời.

ĐIỂM CẦN NHỚ

Một chunk có thể mang metadata:

page: 67
content_group: trinh_bay

Đóng rồi mở lại database vẫn truy xuất được.

LỜI TRÌNH BÀY

Qdrant là kho vector của bài. Em không chỉ lưu vector mà còn lưu text và metadata để hiển thị bằng chứng. Database nằm trên ổ đĩa nên dữ liệu vẫn còn sau khi tắt chương trình. Khi nạp lại, ID ổn định giúp không sinh bản ghi trùng; khi nguồn thay đổi, em refresh dữ liệu để loại các chunk cũ.

7. Ba cấu hình A, B và C khác nhau thế nào?

Cùng corpus, model và benchmark; thay đổi cách chia hoặc cách lọc để so sánh.

A — Chia cố định

Tối đa 700 ký tự; hai chunk liền nhau trùng 100 ký tự. Dễ cắt giữa từ hoặc câu.

B — Ưu tiên đoạn và câu

Tối đa 700 ký tự, không overlap. Gom đơn vị theo ranh giới cấu trúc khi có thể.

C — Như B, thêm bộ lọc

Dùng cùng chunk và vector B; chỉ lọc nhóm nội dung khi được cung cấp trước truy xuất.

ĐIỂM CẦN NHỚ

A–B so sánh toàn bộ công thức chia: cả ranh giới và overlap.

B–C chỉ khác bộ lọc. Nếu không chọn nhóm, C cho kết quả như B.

LỜI TRÌNH BÀY

Ở A, em chia theo số ký tự và dùng overlap để giữ một phần ngữ cảnh giữa hai đoạn. Ở B, em ưu tiên điểm kết thúc câu hoặc đoạn để giữ trọn ý hơn. C dùng cùng dữ liệu như B nhưng thêm bộ lọc nhóm. Việc giữ các yếu tố khác giống nhau giúp em phân tích ảnh hưởng của chiến lược dữ liệu.

8. Bộ lọc metadata hoạt động ra sao?

Lọc là giới hạn phạm vi ứng viên trước khi trả về các đoạn gần câu hỏi nhất.

Không lọc: toàn bộ 62 trang

Hỏi “Có những yêu cầu nào?” có thể gặp yêu cầu sơ chế, làm chín hoặc trình bày.

Lọc nhóm trình bày: trang 65–70

C chỉ xét các chunk có content_group = trinh_bay, rồi xếp hạng theo độ tương đồng.

Phạm vi phải có trước

Người dùng chọn nhóm theo nhu cầu; không lấy trang đáp án trong benchmark để lọc.

ĐIỂM CẦN NHỚ

Bộ lọc giúp giảm kết quả ngoài phạm vi.

Nhưng trong một nhóm vẫn có nhiều đoạn khác nhau, nên hệ thống vẫn có thể chọn sai bằng chứng.

LỜI TRÌNH BÀY

Ví dụ khi em muốn tìm yêu cầu trình bày món ăn, em chọn nhóm trình bày. Qdrant chỉ xét các chunk thuộc nhóm đó, thay vì tìm cả tài liệu. Điều này giúp giảm nhiễu ngoài phạm vi. Tuy nhiên, lọc đúng nhóm chưa đủ, vì model vẫn phải tìm đúng đoạn trong nhóm đã chọn.

9. Benchmark, dev, test và Hit@3

Benchmark là bộ câu hỏi có bằng chứng chuẩn để đo chất lượng một cách nhất quán.

20 câu hỏi có gold evidence

Mỗi câu có đáp án kỳ vọng, trang nguồn và đoạn bằng chứng đã kiểm tra trong text.

15 dev → chọn; 5 test → kiểm tra

Chọn cấu hình bằng dev, cố định lựa chọn rồi mới đánh giá test. Không huấn luyện lại model.

Hit@1 và Evidence Hit@3

Hit@1 xét đoạn đầu. Hit@3 xét có ít nhất một đoạn trong top-3 đúng nguồn/trang và đủ evidence.

ĐIỂM CẦN NHỚ

Có 3 câu dùng filter, trong đó 1 câu thuộc test.

Chốt hash câu hỏi/cấu hình trước thử nghiệm. Không sửa chiến lược dựa trên kết quả test.

LỜI TRÌNH BÀY

Để đánh giá, em xây 20 câu hỏi kèm đáp án và bằng chứng. Em dùng 15 câu dev để chọn cấu hình, sau đó dùng 5 câu test để kiểm tra. Với Hit@3, một câu được tính đúng khi ít nhất một trong ba chunk đầu đúng nguồn, đúng trang và chứa đủ bằng chứng. Chỉ trả về đúng trang nhưng thiếu ý vẫn chưa được tính hit.

10. Kết quả thực nghiệm và lựa chọn

Bảng dưới là Evidence Hit@3 từ lần chạy thật; không thay lựa chọn sau khi xem test.

Cấu hình	Dev Hit@3	Test Hit@3
A	4/15 · 26.7%	2/5 · 40%
B	6/15 · 40.0%	1/5 · 20%
C · chọn	**7/15 · 46.7%**	**1/5 · 20%**

Tập test nhỏ: một câu = 20 điểm phần trăm.

ĐIỂM CẦN NHỚ

Chọn C vì dev tốt nhất: 7/15.

Test của C chỉ 1/5, thấp hơn A 2/5. Giữ kết quả này để phân tích, không chọn lại theo test.

LỜI TRÌNH BÀY

Trên tập dev, C đạt 7 trên 15 câu, cao nhất trong ba cấu hình nên được chọn. Sau khi chốt, C đạt 1 trên 5 câu test, trong khi A đạt 2 trên 5. Em vẫn giữ lựa chọn C theo dev. Tập test chỉ có năm câu, mỗi câu tương đương 20 điểm phần trăm nên chưa thể kết luận cấu hình nào tốt nhất một cách tổng quát.

11. Audit: xem chất lượng ngoài điểm Hit@3

Điểm số cho biết kết quả; audit giúp hiểu vì sao hệ thống đúng hoặc sai.

Coherence — chunk có trọn ý không?

10 vị trí nguồn × 3 cấu hình = 30 mẫu, xem có cắt câu hoặc thiếu chủ thể không.

Metadata — nguồn và nhóm có đúng không?

5 câu test × top-3 × 3 cấu hình = 45 bản ghi đối chiếu với dữ liệu nguồn trích xuất.

Grounding — đủ bằng chứng để trả lời chưa?

5 câu test × 3 cấu hình = 15 trường hợp xem top-3; chấm 0, 1 hoặc 2 và ghi lý do.

ĐIỂM CẦN NHỚ

Audit định tính hiện do AI hỗ trợ và còn chờ học viên xác nhận.

Không gọi nhãn AI là chấm tay của học viên; không coi grounding là độ chính xác câu trả lời LLM.

LỜI TRÌNH BÀY

Ngoài retrieval, em xem chunk có trọn ý không, metadata có đúng không và bằng chứng có đủ để trả lời không. Phần này gọi là audit. Ví dụ một đoạn tìm đúng chủ đề nhưng thiếu điều kiện cần thiết thì grounding vẫn chưa tốt. Các nhận xét định tính hiện do AI hỗ trợ, em chưa xác nhận chấm thủ công nên báo cáo ghi rõ trạng thái này.

12. Một trường hợp thất bại để giải thích

Câu q17: trong phần gia vị, cỏ xạ hương được dùng cho những nhóm món nào?

Bằng chứng cần tìm: trang 40

Nêu cách dùng cỏ xạ hương cho nước dùng, món có mùi hôi và món chế biến nhiệt dài.

Kết quả C: trang 39, 39 và 38

Đúng nhóm gia vị, nhưng các đoạn nói về mùi tây, hương thảo và những gia vị khác.

Kết luận từ lỗi

Filter tuân thủ phạm vi nhưng retrieval vẫn nhầm thực thể. Có thể nghiên cứu hybrid/reranker ở vòng mới.

ĐIỂM CẦN NHỚ

Không sửa mô hình hoặc bộ câu hỏi hiện tại để tăng điểm sau khi đã xem test.

Nếu thử cải tiến, chọn trên dev và cần tập test phù hợp cho vòng đánh giá mới.

LỜI TRÌNH BÀY

Một lỗi cụ thể là câu hỏi về cỏ xạ hương. Đáp án nằm ở trang 40, nhưng cấu hình C trả về trang 39 và 38, nói về các loại gia vị khác. Bộ lọc đã làm đúng nhiệm vụ giới hạn nhóm, nhưng model vẫn nhầm đối tượng. Đây là lý do cần đọc trace, thay vì chỉ nhìn thấy đúng chủ đề rồi kết luận truy xuất thành công.

13. Demo giao diện và kết luận

Chạy: .\.venv\Scripts\python.exe -m starter.src.webapp --open

Bước 1 — Tìm một câu hỏi

Chọn B; nhập "Gia vị có vai trò gì trong món ăn?"; bấm Tìm bằng chứng.

Bước 2 — Kiểm tra nguồn

Đọc top-3, xem điểm cosine, bấm số trang để đối chiếu PDF; điểm cao chưa chắc đủ ý.

Bước 3 — So sánh

Chọn nhóm phối hợp nguyên liệu/gia vị và bật so sánh A/B/C. Bộ lọc chỉ áp dụng cho C.

ĐIỂM CẦN NHỚ

Đã có pipeline chạy thật, index lưu bền vững và 6 kiểm thử kỹ thuật đạt.

Điều học được: giữ lineage, kiểm soát chất lượng, nạp lại không trùng và đánh giá bằng bằng chứng.

LỜI TRÌNH BÀY

Cuối cùng em demo một câu hỏi trên giao diện, đọc ba đoạn truy xuất và mở trang PDF để kiểm tra. Sau đó em so sánh A, B, C với cùng câu hỏi. Qua bài này, em thực hành cách tổ chức pipeline, giữ liên kết về nguồn và kiểm chứng kết quả. Hạn chế còn lại là lỗi text PDF, model nhỏ, test ít câu và phần nhãn thủ công chưa xác nhận. Em xin kết thúc phần trình bày.